

The goal of this design is to make Compass's AI registry more secure, controlled, and defensible. Compass should use Microsoft Purview's supported AI sites list as a trusted starting source, but it should not treat that list as the only source of truth and it should not allow an import script to directly modify the live approved registry. Microsoft's Purview documentation says the supported AI sites article lists generative AI sites supported by Purview Data Security Posture Management and DSPM for AI, and Microsoft also notes that the list is expected to increase over time. That makes it useful as a seed list, but it also means Compass needs a process for tracking changes over time. ([Microsoft Learn](#))

The first thing to understand is the separation between GitHub and Azure. GitHub should store the project files, documentation, SQL setup scripts, import scripts, sample data, and setup instructions. Azure should store the actual live database, schema, tables, roles, permissions, registry records, ledger history, and approval records. GitHub is the blueprint. Azure SQL Database is the actual working system. This distinction is important because your instructor should be able to see how the system is built by reviewing the GitHub repository, but your live registry should not be editable from GitHub or from the frontend application.

Start by creating a GitHub repository for this part of the Compass project. The repository should include folders for infrastructure scripts, import scripts, sample data, and documentation. The SQL files should be stored in a folder such as `infra/sql`, the Python import script should be stored in a folder such as `scripts`, and the written explanation should be stored in a folder such as `docs`. The GitHub repository should not contain real passwords, database connection strings, private keys, Azure credentials, or real `.env` files. It is acceptable to include a `.env.example` file that shows what environment variables are needed, but the real `.env` file should be excluded using `.gitignore`.

The Azure side begins with creating an Azure SQL Database. For a class project, the Azure SQL Database free offer is likely enough because the registry is small. Microsoft's free offer documentation states that each free-offer database includes 100,000 vCore seconds, 32 GB of data storage, and 32 GB of backup storage per month, with up to 10 General Purpose databases per subscription. ([Microsoft Learn](#)) The Compass AI registry will only contain domain names, metadata, risk levels, source information, and approval records, so it should be far below the storage limit. For a class project, the realistic cost can be zero dollars if configured carefully, but it is still smart to set billing alerts and avoid enabling paid overage unless you intentionally want that.

When creating the Azure SQL Database, enable ledger functionality. Azure SQL Ledger is useful here because the main security concern is not only confidentiality, but integrity. In other words, the biggest risk is not someone reading the registry. The bigger risk is someone modifying it so Compass no longer detects a risky AI tool or incorrectly marks an unsafe tool as approved. Microsoft's ledger quickstart explains how to create a ledger database in Azure SQL Database and configure automatic digest storage through the Azure portal. ([Microsoft Learn](#))

Ledger gives you tamper-evident history, which helps you prove whether security-critical registry data was altered.

After creating the database, configure automatic digest storage. Ledger digests are cryptographic checkpoints that support later verification of the ledger history. Microsoft documents that automatic digest storage can be configured for Azure SQL Database through the Azure portal, PowerShell, or Azure CLI. ([Microsoft Learn](#)) For the Compass project, you can keep this simple by creating an Azure Storage account and letting Azure SQL store ledger digests there. For a stronger design, configure immutability on the digest storage container so that the digest records are also protected from deletion or modification. This gives you a stronger answer when explaining how the system detects tampering.

Once the database exists, create the database schema. The schema is a namespace inside Azure SQL Database. For this project, use the schema name `compass`. The SQL statement to create it should be saved in GitHub inside a file such as `infra/sql/001_create_schema.sql`, but the statement itself must be executed against Azure SQL Database. After execution, the actual `compass` schema exists in Azure, while the SQL file remains in GitHub as documentation and repeatable setup code.

Next, create the registry tables in Azure SQL Database. These table definitions should also be stored as SQL files in GitHub, but the actual tables should live in Azure. The three core tables are `registry_import_run`, `pending_ai_registry`, and `approved_ai_registry`. The `registry_import_run` table records each time Compass imports or checks the Microsoft Purview supported AI sites list. It should store the source name, source URL, source hash, number of domains found, who imported it, and the timestamp. This matters because it creates an audit trail showing when the external source was checked and what version of the source was used.

The `pending_ai_registry` table is where new or changed AI domains should go before approval. The import script should only be allowed to insert into this table. It should not be allowed to directly insert into the approved registry. This is a key security design decision. If the import script were compromised, the attacker should not be able to approve a malicious or unsafe domain automatically. The pending table should include fields such as domain pattern, normalized domain, source name, change type, suggested tool name, suggested category, suggested risk level, review status, reviewer notes, requested timestamp, reviewer name, and reviewed timestamp.

The `approved_ai_registry` table is the official registry that Compass uses for detection. This table should only contain domains that have been reviewed and approved. The Compass detection engine should read from this table only. It should not read directly from the pending table and it should not read directly from the Microsoft Purview page at runtime. The approved table should include fields such as domain pattern, normalized domain, tool name, category, risk level, source name, approval status, approved by, approved at, and active status. The active status is useful because a domain can later be disabled without deleting its historical record.

The reason for using both pending and approved tables is that Microsoft Purview gives Compass a useful external reference list, but Compass still needs a company-specific governance decision. Microsoft's list can tell you that a site is part of a supported AI-site list, but it does not decide whether a specific company should approve, restrict, monitor, or block that tool. Compass should add that organizational context. For example, a domain such as chatgpt.com may be recognized as a known AI site, but the organization still needs to decide whether it is allowed, allowed only with restrictions, monitored, or blocked.

After creating the tables, create stored procedures for approval and rejection. These stored procedures should also be stored in GitHub but executed in Azure SQL Database. Reviewers should not manually insert into the approved table. Instead, they should execute a stored procedure that moves a pending record into the approved registry and updates the pending record's review status. This makes the approval process more controlled and easier to audit. The approval procedure should require the pending record ID, final tool name, category, risk level, and approver name. The rejection procedure should require the pending record ID, reviewer name, and reviewer notes.

The approval procedure should first confirm that the pending record exists and is still pending. If the record is not found or has already been reviewed, the procedure should stop and return an error. If the record is valid, the procedure inserts it into the approved registry and updates the pending record to show that it was approved. This gives you a clear history of the decision. The rejection procedure should not delete the record. Instead, it should mark the pending record as rejected and store the reason. Rejected records are useful because they show that the security team reviewed the domain and decided not to include it.

Next, create separate database roles. This is one of the most important security steps. The import script, security reviewer, and Compass application should not all use the same database account. The import script should have permission to insert import runs and pending records, and it may need permission to read the approved registry so it can avoid creating duplicate pending records. The reviewer should have permission to view pending records and execute the approval or rejection stored procedures. The Compass application should only have permission to read the approved registry. The Compass application should not have insert, update, delete, or approval permissions.

This role separation protects the registry if part of the application is compromised. If an attacker compromises the frontend or the normal Compass detection API, they should only be able to read the approved registry, not modify it. If an attacker compromises the import script, they should only be able to create pending review items, not approve them. If an attacker wants to change the live registry, they would need access to the approval role or a privileged database role, and ledger history would still help detect suspicious changes.

After the database structure is ready, create the Python import script. The script should live in GitHub under the [scripts](#) folder. Its job is to pull the Microsoft Purview supported AI sites page, extract domain patterns, normalize them, calculate a hash of the source list, compare the domains against the approved registry and existing pending records, and insert only new

domains into the pending table. The script should not approve anything. It should only stage findings for review. This design makes the import script safe to run repeatedly because it should avoid duplicating records that are already approved or already pending.

The import script should use environment variables for the Azure SQL connection settings. Do not hardcode the server name, database name, username, or password directly into the Python file. Use a `.env.example` file in GitHub to show the expected configuration values, but keep the real `.env` file only on your local machine or in a secure secret store. The `.gitignore` file should explicitly exclude `.env`, Python cache files, and any generated files that may contain sensitive values. For a production-style design, credentials would be stored in a managed secret service, but for the class project, environment variables are acceptable as long as the real values are not pushed to GitHub.

When the script runs, it should first download the Microsoft Purview page. It should then extract wildcard domain patterns, such as domains beginning with `*..` It should normalize each domain by converting it to lowercase and removing the wildcard prefix when storing the normalized value. For example, `*.chatgpt.com` would have a domain pattern of `*.chatgpt.com` and a normalized domain of `chatgpt.com`. Keeping both values is useful because the pattern is useful for matching subdomains, while the normalized domain is useful for comparison and deduplication.

The script should calculate a SHA-256 hash of the sorted source domain list. This hash should be stored in `registry_import_run` along with the source URL and timestamp. The purpose of the hash is to create a record of the exact source list used during that import run. If Microsoft's list changes later, the hash will change. This helps show that Compass is not just storing random domain records; it is tracking versions of an external reference source.

After calculating the source hash, the script should query the approved registry and the pending registry. It should check whether each normalized domain already exists as an active approved record or as an existing pending record. If a domain is already approved, the script should skip it. If a domain is already pending, the script should skip it. If the domain is not found in either place, the script should insert it into the pending table with a review status of `Pending`, a change type such as `ADD`, and a suggested risk level such as `Medium` or `Medium-High`. The suggested risk is only a starting point. The reviewer makes the final decision.

Once the import script has staged records in the pending table, the reviewer should examine them. In a full Compass application, this would be done through an admin review page. For the class project, it can be done using Azure Data Studio, SQL Server Management Studio, or the Azure portal query editor. The reviewer should inspect the pending domain, source, suggested category, suggested risk level, and any notes. The reviewer should then decide whether to approve, approve with restrictions, monitor, block, reject, or mark as a false positive.

For known Microsoft Purview supported AI sites, the approval process may be faster because the source is trusted. However, trusted source does not mean automatic approval. The

organization still needs to decide whether that tool is acceptable for its users and data. Some AI tools may be safe for general use but not safe for sensitive data. Some may be approved only for enterprise accounts. Some may be blocked because they do not meet the company's security or privacy standards. Compass should capture that decision in the approved registry.

The next part of the design is handling unknown AI usage. This is necessary because employees may use AI tools that are not listed in Microsoft Purview's supported AI sites list. The correct design is not to ignore those tools. Instead, Compass should classify them as unknown AI usage and stage them for review. This makes the system stronger because it does not depend entirely on Microsoft's list. Microsoft Purview provides a strong seed list, but logs from the organization show what users are actually doing.

Unknown AI usage should come from DNS logs, proxy logs, firewall logs, browser telemetry, or other network sources available to the organization. When Compass parses those logs and finds a domain that looks like AI usage but does not match the approved registry, it should create a pending registry item with a change type such as **ADD_UNKNOWN**. The record should include where it was discovered, when it was first seen, when it was last seen, how many users accessed it, how often it was accessed, whether data uploads were observed, and what log source detected it. This gives the reviewer enough context to make a decision.

For unknown AI tools, Compass should assign a temporary elevated risk level. A good default is **Medium-High** for unknown AI usage and **High** if there is evidence of file uploads, large data transfer, sensitive department usage, many users, suspicious ownership, or unclear vendor identity. This does not mean every unknown AI tool is malicious. It means the company has not reviewed it yet, so Compass should treat it cautiously. This is a good security posture because unknown AI tools could create data leakage, privacy, compliance, or intellectual property risks.

The reviewer should enrich unknown AI records before approving them. The reviewer should determine whether the vendor is legitimate, whether the tool has a clear privacy policy, whether submitted data can be used for model training, whether the tool supports enterprise controls, whether it supports SSO, whether it supports data retention settings, whether it has security documentation such as SOC 2 or ISO 27001, and whether there is a real business need. The reviewer should also look at usage context, such as which users or departments accessed the tool and whether uploads or sensitive activity occurred.

After review, unknown AI usage can have several outcomes. If the tool is legitimate and acceptable, it can be approved and added to the approved registry. If the tool is useful but risky, it can be approved with restrictions, such as allowing access only for certain users or warning users not to paste sensitive data. If there is not enough information, it can remain on a watchlist. If the tool is risky or unauthorized, it can be marked as blocked and sent to the firewall, proxy, DNS filter, or security policy team for enforcement. If the domain is not actually an AI tool, it can be marked as a false positive.

This unknown AI usage workflow is important for your presentation because it shows that Compass is not simply copying Microsoft's list. Compass uses Microsoft Purview as a trusted

reference source, but it also discovers company-specific AI usage from real logs. This allows Compass to identify shadow AI tools that may not appear in the official seed list. Microsoft's Purview materials also discuss discovering and monitoring AI app usage to prevent data leakage to shadow AI, which supports the idea that AI governance needs both known-site coverage and discovery of actual usage. ([Microsoft Learn](#))

The Compass detection engine should use the approved registry as its trusted lookup table. It should parse DNS, proxy, and firewall logs and compare observed domains against the approved AI registry. If the observed domain matches an approved registry entry, Compass should classify the activity as known AI usage and apply the stored metadata, such as tool name, category, risk level, and recommended action. If the observed domain does not match the approved registry but looks AI-related or is flagged by discovery logic, Compass should classify it as unknown AI usage and stage it in the pending registry.

The report should clearly distinguish between known AI tools and unknown AI tools. Known AI tools are those that match the approved registry. Unknown AI tools are those detected from logs but not approved yet. The report should not treat unknown as automatically malicious, but it should treat unknown as needing review. A strong report summary might say that Compass detected a certain number of known AI tools, a certain number of unknown AI tools, and a certain number of high-risk unknown tools. It should also list recommended next actions, such as approve, restrict, monitor, block, or reject.

For the user interface, the Security Lead view should show the pending review queue. It should allow the reviewer to see the domain, source, first seen time, last seen time, user count, request count, data volume, temporary risk level, and recommended action. The reviewer should then be able to approve, reject, monitor, or block the tool. The CEO or executive view should avoid database and framework language. It should show business impact, such as unapproved AI usage, sensitive data exposure risk, top risky tools, and recommended actions. The team or operational view should show which domains were detected and what policy applies to each one.

The approved registry should also support versioning. Each approved release of the registry should have a version identifier, timestamp, record count, and hash. This makes scan results more defensible. For example, if Compass generates a report, the report should state which registry version was used. That way, if the registry changes later, the older report is still understandable. This is useful for audits, class presentations, and incident reviews.

For extra protection, you can export the approved registry to a signed JSON file after approval. Azure SQL Database remains the source of truth, but a signed JSON export can make the detection engine faster or easier to demo. The JSON file should include the registry version, generated timestamp, record count, records, and SHA-256 hash. The signature should be verified before the detection engine uses the file. This is optional for the class project, but it is a strong design feature because it shows that even exported registry data is protected against tampering.

The database should also use encryption and secure access controls. Azure SQL Database includes security features such as encryption at rest, but for this specific registry, integrity matters more than secrecy. The domain registry itself may not be highly confidential, but if an attacker changes it, Compass's results could become misleading. Therefore, the most important controls are ledger tables, digest storage, role-based access, stored procedures, read-only application access, and audit history.

The class project should include sample data so the system can be demonstrated without relying entirely on live cloud resources. In GitHub, include a sample Purview domain list, sample pending registry records, sample approved registry records, and sample detection results. These should be clearly labeled as demo data. This lets your instructor understand the workflow even if they do not run the Azure database themselves. The live Azure SQL Database still represents the secure implementation.

Your README should explain the setup in a clear order. First, explain the purpose of the secure AI registry. Second, explain that Microsoft Purview supported AI sites are used as a seed source. Third, explain that all imported domains go to a pending table first. Fourth, explain that a reviewer must approve or reject each domain. Fifth, explain that the approved registry is stored in Azure SQL Ledger tables. Sixth, explain that Compass reads from the approved registry only. Seventh, explain how unknown AI usage is handled when domains appear in company logs but are not in the approved registry. Eighth, explain how the registry is protected from unauthorized modification.

When documenting the security design, emphasize that the import script cannot modify the approved registry, the frontend cannot modify the registry, and the detection engine only has read access. The only path into the approved registry should be through the controlled approval process. This is the central security idea. It prevents automatic trust, reduces the impact of a compromised script or frontend, and creates a reviewable record of every registry decision.

When explaining this to an instructor, you can say that Compass uses Microsoft Purview's supported AI sites as a trusted external seed source, but the company-specific registry is controlled separately. The import script stages new domains in a pending registry table. A security reviewer then assigns the business category, risk level, and final decision. Approved records are stored in Azure SQL Ledger tables so changes are tamper-evident. The detection engine uses a read-only identity and can only query active approved records. Unknown AI tools discovered from logs are treated as elevated-risk pending items until reviewed.

The final implementation order should be handled carefully. First, create the GitHub repository and folder structure. Second, create the Azure SQL Database with ledger enabled. Third, configure automatic digest storage. Fourth, create the `compass` schema. Fifth, create the ledger-backed tables. Sixth, create the stored procedures. Seventh, create database roles and permissions. Eighth, create the Python import script. Ninth, run the import script to stage Microsoft Purview domains. Tenth, review and approve selected records. Eleventh, connect the Compass detection engine to the approved registry using read-only access. Twelfth, test

unknown AI detection by feeding sample DNS, proxy, or firewall logs containing domains that are not already approved.

For the MVP, you do not need to build every enterprise feature. The minimum strong version is an Azure SQL Ledger database, three registry tables, approval stored procedures, separate database roles, a Python import script, sample DNS or proxy logs, and a basic review process. The stretch version adds a review dashboard, signed JSON exports, registry versioning, unknown AI enrichment, and automated recommendations. This gives you a realistic but achievable class project.

The most important thing to remember is that Compass should never blindly trust one source or one script. Microsoft Purview is the starting point. Azure SQL Ledger is the integrity layer. The pending table is the safety gate. The reviewer is the governance step. The approved registry is the trusted detection source. The unknown AI workflow is how Compass finds tools outside the known list. Together, these pieces create a secure and explainable AI registry system for the project.